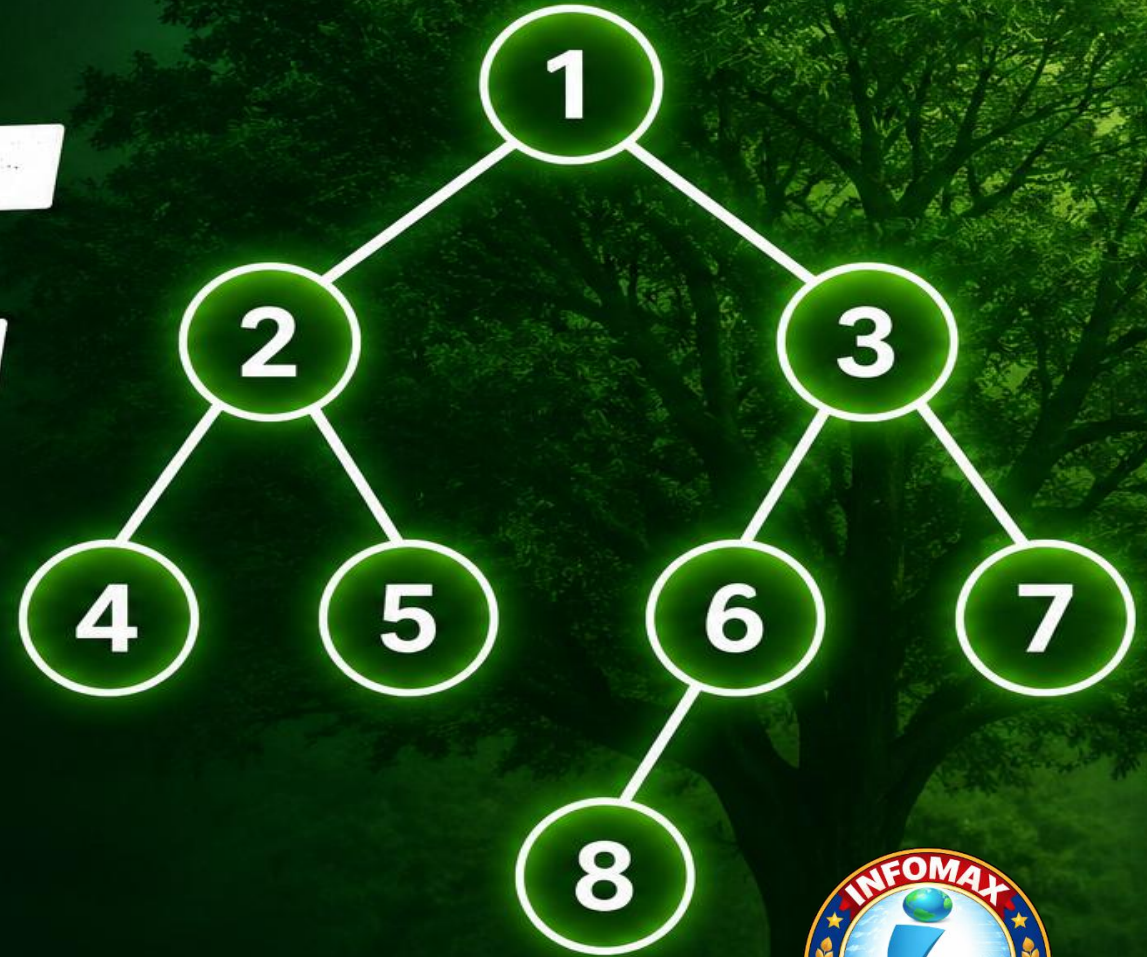


DSA DATA STRUCTURES & ALGORITHMS

TREE

THE FOUNDATION OF
HIERARCHY IN DSA



EASY TO
UNDERSTAND



CRUCIAL FOR
INTERVIEWS



ACE YOUR
DSA JOURNEY



Introduction to Tree

➤ What is a Tree?

- A Tree is a **non-linear hierarchical data structure** consisting of nodes connected by edges.

➤ Features

- Stores hierarchical data
- Faster searching and sorting
- Represents parent-child relationships

➤ Real-Life Examples

- File system
- Organization hierarchy
- Family tree
- HTML DOM structure

➤ Components of Tree

- **Node** → Basic element of tree
- **Edge** → Connection between nodes
- **Root Node** → Topmost node
- **Child Node** → Node below another node
- **Parent Node** → Node above child node
- **Leaf Node** → Node with no children

➤ Example



Term	Description
Root	Starting node
Parent	Node having child
Child	Descendant node
Sibling	Nodes with same parent
Leaf	Node without children
Internal Node	Node with at least one child
Degree	Number of children
Height	Longest path from root
Level	Position of node

➤ Properties

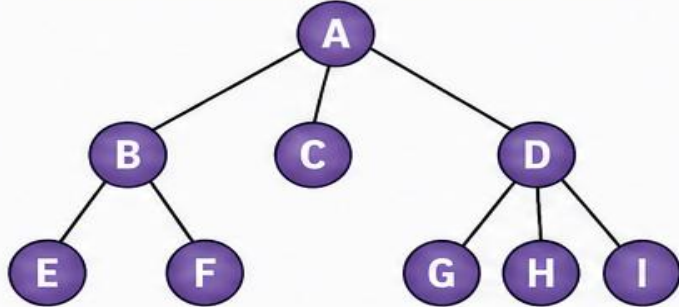
- A tree with n nodes has $n-1$ edges.
- Only one path exists between two nodes.
- Trees do not contain cycles.
- Root node has no parent.

Types of Trees

- General Tree
- Binary Tree
- Binary Search Tree (BST)
- AVL Tree
- B-Tree

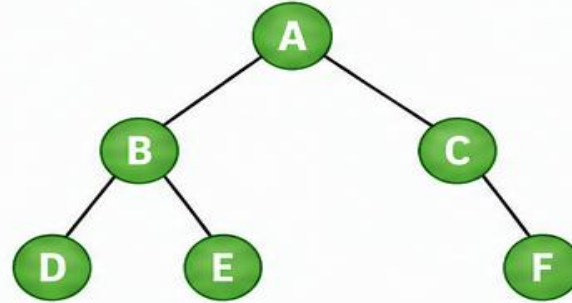
TYPES OF TREES IN DSA

1. GENERAL TREE



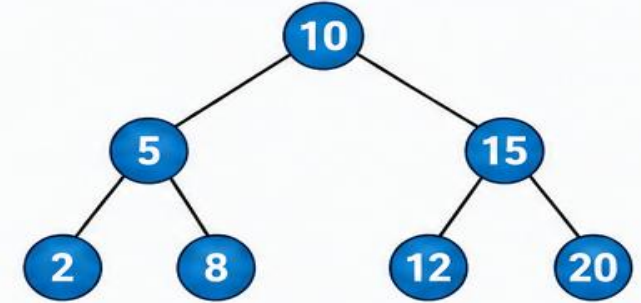
- A tree in which a node can have any number of children.
- No restriction on the number of children.

2. BINARY TREE



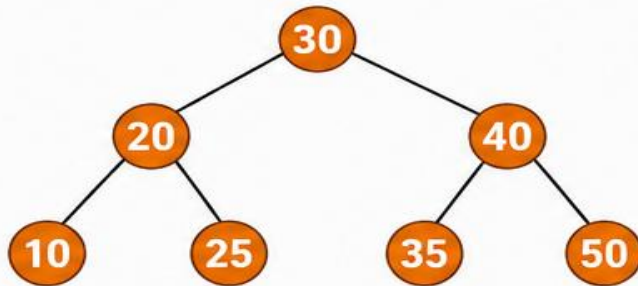
- A tree in which a node can have at most two children.
- Children are typically called Left and Right.

3. BINARY SEARCH TREE (BST)



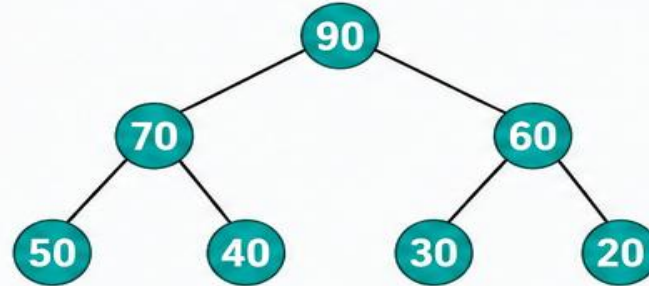
- For any node, all values in the left subtree are smaller.
- All values in the right subtree are larger.

4. AVL TREE



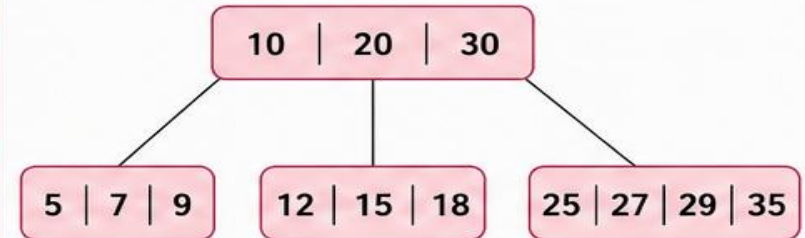
- A self-balanced Binary Search Tree.
- For every node, the height difference between left and right subtree is at most 1.

5. HEAP TREE



- A Complete Binary Tree.
- Follows Heap Property (Max-Heap or Min-Heap).
- In Max-Heap, parent is greater than children.

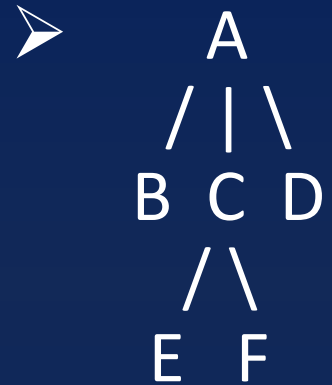
6. B-TREE



- A self-balanced tree with multiple keys per node.
- Each node can have multiple children.
- Keys in a node are sorted.
- Used in databases and file systems.

➤ A tree where a node can have any number of children.

➤ **Example**



GENERAL TREE

A tree in which a node can have any number of children.

1. DEFINITION

A General Tree is a hierarchical data structure in which each node can have **zero or more** children.

2. KEY POINTS



Each node can have **any number** of children.



There is only **one root** node.



There is **no restriction** on the number of children.

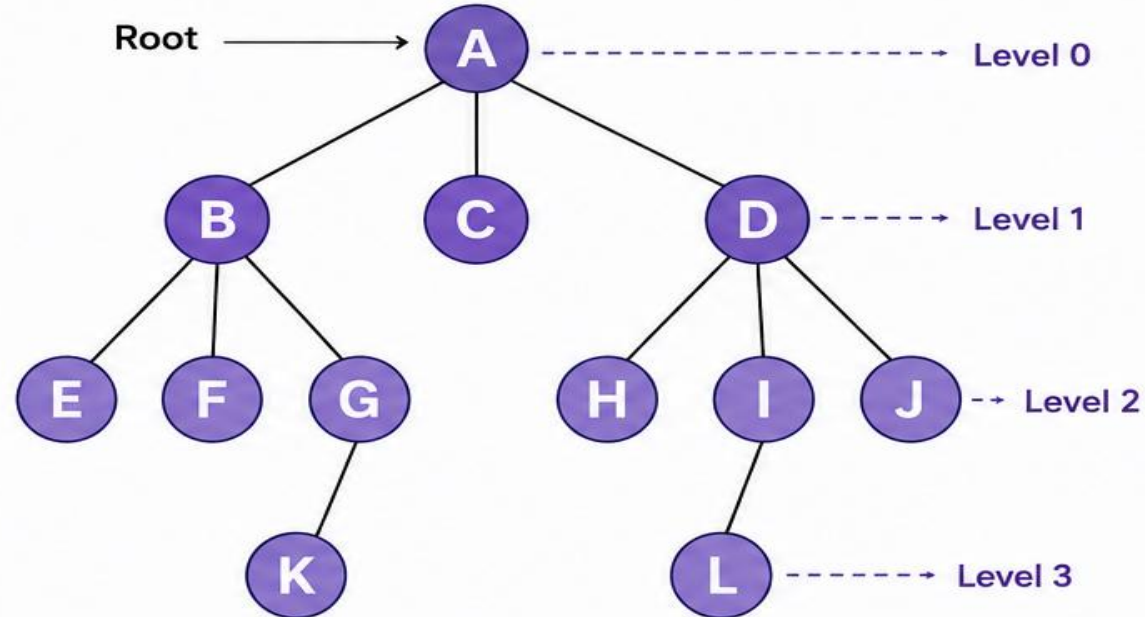


A node can have **zero** children (leaf node).



It **does not** follow any binary property.

3. EXAMPLE



— Parent → Child

● Node

- - - Level

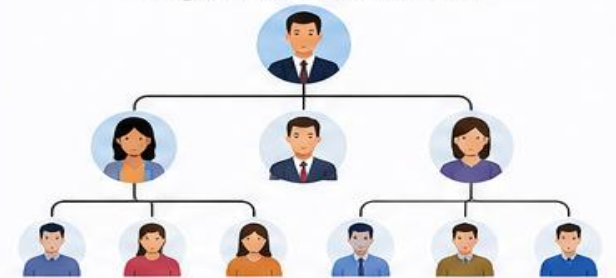
○ Leaf Node (No Children)

4. PROPERTIES

- There is exactly one root.
- Each node can have **0 or more** children.
- If a node has no children, it is called a **leaf node**.
- The maximum number of children is not fixed.
- The number of edges in a general tree with N nodes is $N - 1$.

5. REAL-LIFE ANALOGY

Organization Structure



TRAVERSALS
(Same as other trees)

- **Preorder** (Root, Children)

A B E F G C D H I L J

- **Postorder** (Children, Root)

E F K G B C H L J D A

- **Level Order** (BFS)

A B C D E F G H I J K L

➤ A Binary Tree is a tree in which each node can have at most two children.

➤ **Children**

➤ Left Child

➤ Right Child

➤ **Example**





BINARY TREE



1. DEFINITION

A Binary Tree is a hierarchical data structure in which each node has at most two children: a left child and a right child.

2. KEY POINTS



Each node can have 0, 1 or 2 children.



Children are ordered: Left child and Right child.



The left and right subtrees are also binary trees.

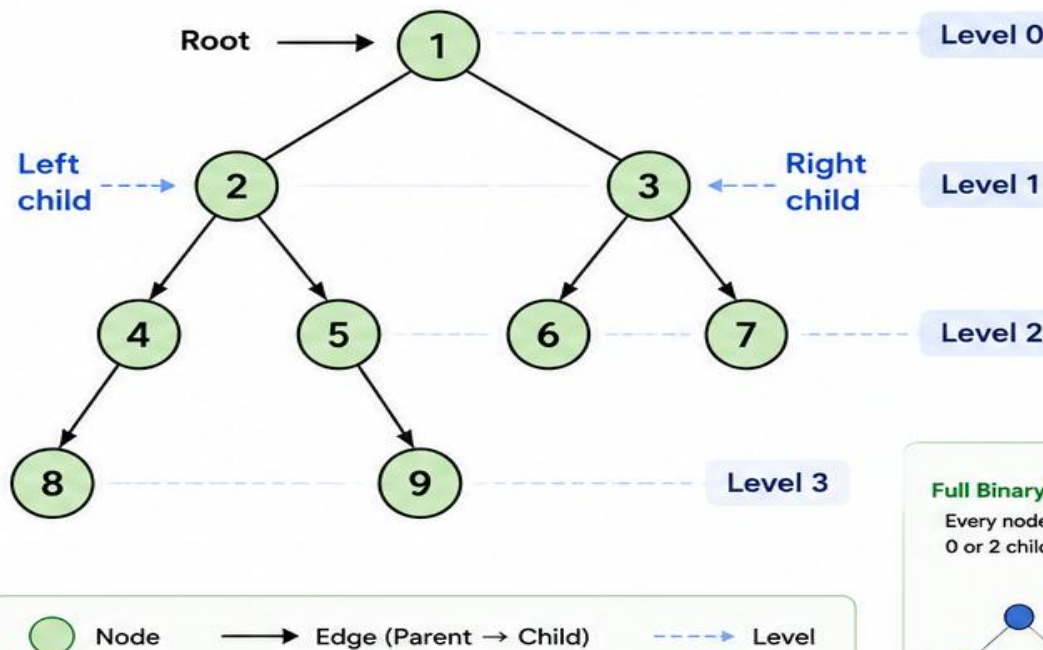


The number of nodes at level i is at most 2^i .



The maximum number of nodes in a tree of height h is $2^{h+1} - 1$.

3. EXAMPLE



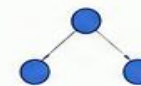
4. PROPERTIES

- Each node has at most two children.
- The left and right subtrees of any node are also binary trees.
- If a node has only one child, it is called a **single child**.
- In a binary tree with N nodes, the number of edges is $N - 1$.
- The i^{th} level (0-indexed) has at most 2^i nodes.

5. TYPES OF BINARY TREE

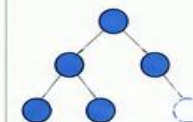
Full Binary Tree

Every node has 0 or 2 children.



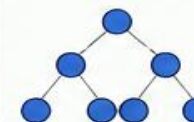
Complete Binary Tree

All levels are full except possibly the last, filled from left to right.



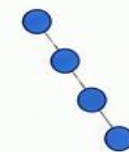
Perfect Binary Tree

All internal nodes have 2 children and all leaves are at the same level.



Skewed Binary Tree

Each node has only one child.



6. TRAVERSALS

• **Preorder** (Root, Left, Right)

1 2 4 8 5 9 3 6 7

• **Inorder** (Left, Root, Right)

8 4 2 5 9 1 6 3 7

• **Postorder** (Left, Right, Root)

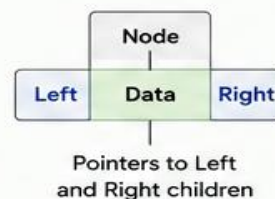
8 4 9 5 2 6 7 3 1

• **Level Order** (BFS)

1 2 3 4 5 6 7 8 9

7. REPRESENTATION

• **Linked Representation**



• **Array Representation**

For complete binary tree (1-indexed)

Index: 1 2 3 4 5 6 7

Tree: 1 2 3 4 5 6 7

For node at index i :

- Left child $\rightarrow 2i$
- Right child $\rightarrow 2i + 1$

8. APPLICATIONS



Used in expression trees, parsing, and syntax analysis.



Used in searching, sorting, and decision making.



Used in heaps, priority queues, and Huffman coding.



Used in file systems, compilers, and AI algorithms.

➤ **1. Full Binary Tree**

➤ Every node has 0 or 2 children.

➤ **2. Complete Binary Tree**

➤ All levels completely filled except last.

➤ **3. Perfect Binary Tree**

➤ All internal nodes have two children and all leaves are at same level.

➤ **4. Skewed Binary Tree**

➤ Nodes arranged in one direction.

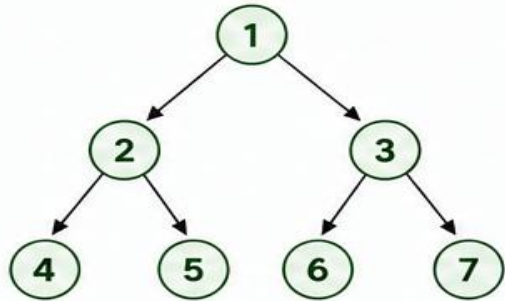


TYPES OF BINARY TREES



1. FULL BINARY TREE

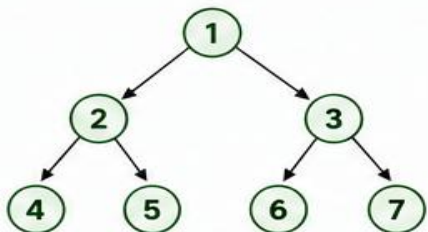
A binary tree in which every node has either **0 or 2 children**.



Key Points

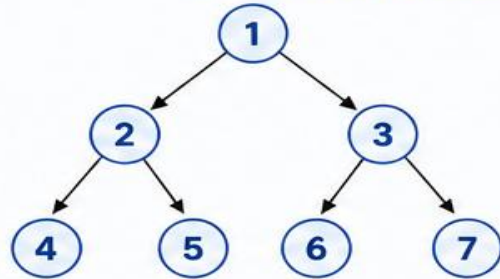
- Every node has 0 or 2 children.
- No node has exactly one child.
- If the number of leaf nodes is L , then total nodes = $2L - 1$.

Example



2. COMPLETE BINARY TREE

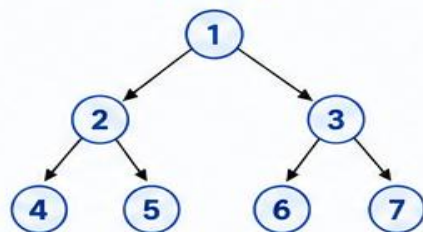
A binary tree in which all levels are completely filled except possibly the last level, and all nodes in the last level are **as far left as possible**.



Key Points

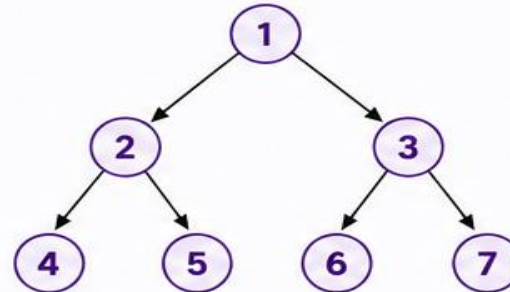
- All levels are full except possibly the last level.
- Last level nodes are placed from **left to right** with no gaps.
- Not all nodes need to have 2 children.

Example



3. PERFECT BINARY TREE

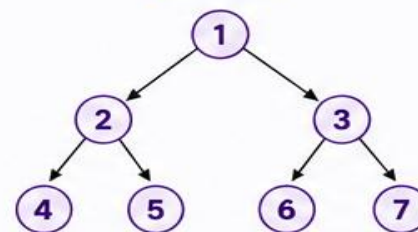
A binary tree in which all internal nodes have 2 children and all leaf nodes are at the **same level**.



Key Points

- All internal nodes have 2 children.
- All leaf nodes are at the same level.
- For height h (levels), total nodes = $2^h - 1$.
- For depth d (edges), total nodes = $2^{d+1} - 1$.

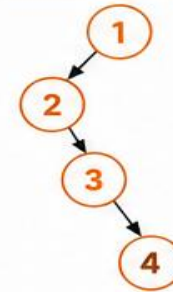
Example



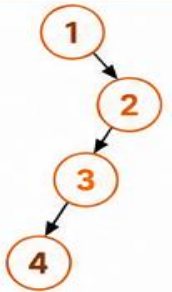
4. SKEWED BINARY TREE

A binary tree in which every node has **only one child**.

Left Skewed



Right Skewed

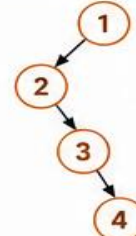


Key Points

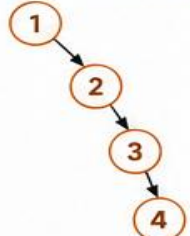
- Each node has exactly one child.
- All nodes form a single chain.
- Number of nodes = height + 1.

Example

Left Skewed



Right Skewed



Node



Edge (Parent → Child)



Level



Note:

All these are types of Binary Trees based on structure and node arrangement.

➤ Binary Search Tree

➤ A BST is a binary tree where:

➤ Left subtree values $<$ Root

➤ Right subtree values $>$ Root

➤ Example





CONSTRUCT BINARY TREE FROM GIVEN VALUES



To construct a Binary Tree from given values, insert the nodes **level by level from left to right**.

1 GIVEN VALUES

Values (Level Order):

1 2 3 4 5 6 7

Array Representation:

[1, 2, 3, 4, 5, 6, 7]

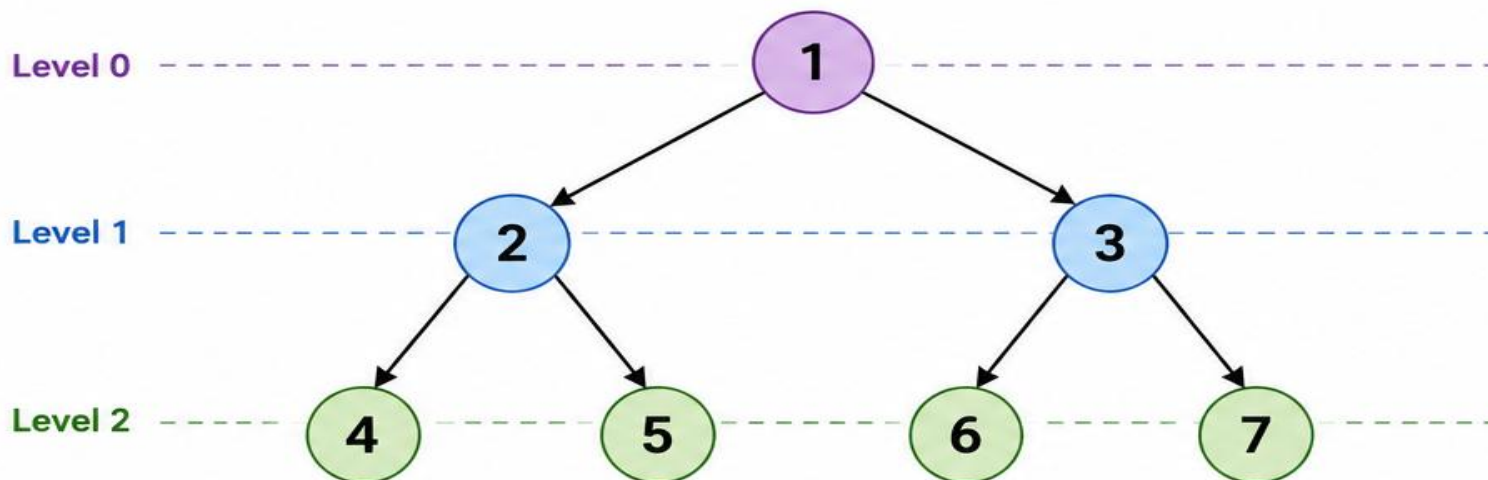
2 LEVEL ORDER CONSTRUCTION

Step	Level	Nodes Inserted	Explanation
1	Level 0	1	1 is the root node.
2	Level 1	2, 3	2 and 3 are inserted as left and right child of 1.
3	Level 2	4, 5, 6, 7	4 and 5 are children of 2. 6 and 7 are children of 3.



Rule: Fill the tree level by level from left to right.

3 CONSTRUCTED BINARY TREE



4 PARENT → CHILD RELATION

- 1 → 2, 3
- 2 → 4, 5
- 3 → 6, 7

5 TYPE OF BINARY TREE

This is a **Perfect Binary Tree** because:

- ✓ Every parent has exactly 2 children.
- ✓ All leaf nodes are at the same level.
- ✓ For height $h = 2$, total nodes = $2^{h+1} - 1 = 7$



Summary: We inserted the given values level by level from left to right to construct the Binary Tree.

Advantages of BST



- Fast searching
- Efficient insertion and deletion
- Maintains sorted data

➤ Traversal means visiting all nodes of a tree.

➤ **Types**

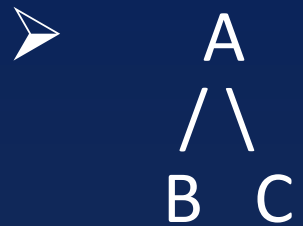
➤ Preorder

➤ Inorder

➤ Postorder

➤ Preorder (Root → Left → Right)

➤ **Example**



➤ **Output**

➤ A B C

➤ **Algorithm**

1. Visit Root
2. Traverse Left
3. Traverse Right

➤ Inorder (Left → Root → Right)

➤ **Output**

➤ B A C

➤ **Algorithm**

1. Traverse Left
2. Visit Root
3. Traverse Right

➤ **Postorder (Left → Right → Root)**

➤ **Output**

➤ B C A

➤ **Algorithm**

1. Traverse Left
2. Traverse Right
3. Visit Root

➤ **1. Array Representation**

- Used in complete binary trees.

➤ **2. Linked Representation**

- Each node contains:
 - Data
 - Left pointer
 - Right pointer


```
➤ struct Node
{
    int data;
    struct Node *left;
    struct Node *right;
};
```

- File systems
- Database indexing
- XML/HTML parsing
- Artificial Intelligence
- Decision trees
- Compiler design
- Network routing